Department of Computer Science and Engineering

Course: CSE447: Cryptography and Cryptanalysis

Semester: Spring 2026

Project Report

Sovereign Guard: Secure System for User Data and Social Posts using RSA/ECC

Submitted To: [Instructor Name]

Group No: [Update]

Section: [Update]

Submission Date: [DD Month YYYY]

No.	Full Name	Student ID
1	[Member 1 Full Name]	[Member 1 Student ID]
2	[Member 2 Full Name]	[Member 2 Student ID]
3	[Member 3 Full Name]	[Member 3 Student ID]

Table of Contents

1. Introduction and System Overview
2. Login and Registration Module
3. User Data Encryption and Decryption
4. Password Hashing and Salting
5. Two-Factor Authentication (2FA)
6. Key Management Module
7. Post and Profile Management
8. Data Storage Security
9. Message Authentication Code (MAC)
10. Role-Based Access Control (RBAC)
11. Secure Session Management
12. GitHub Repository and Project Structure
13. Conclusion

1. Introduction and System Overview

Sovereign Guard is a secure browser-based web application for encrypted account management and encrypted social posts. It demonstrates how user data, profile records, posts, key metadata, and session tokens can remain protected even if raw storage is inspected.

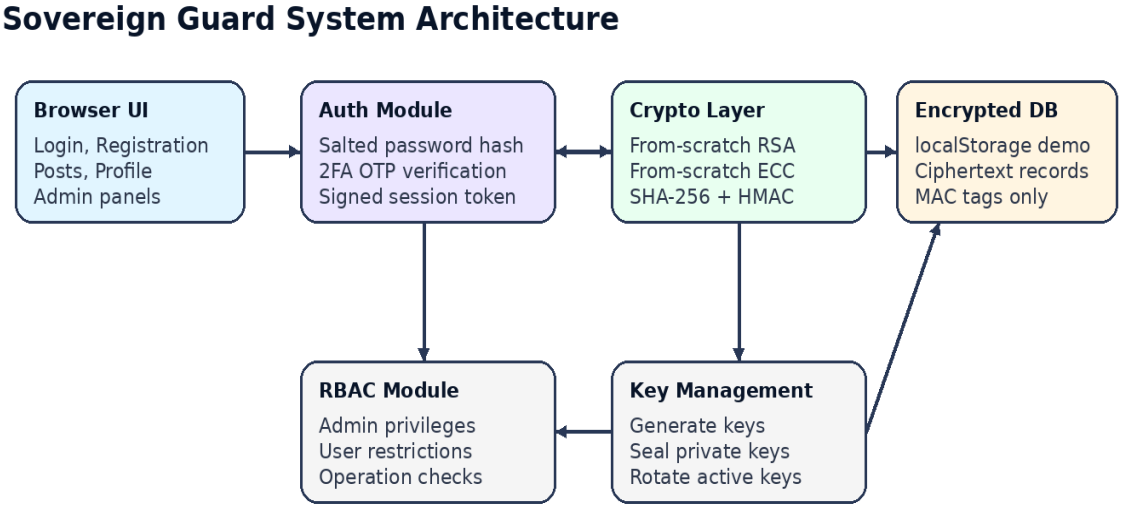

Figure: 1. Introduction and System Overview screenshot/diagram.

1.1 Project Overview

Users can register, log in with two-factor verification, manage profiles, and create or edit posts. Administrators can inspect user summaries, rotate keys, view RBAC controls, and manage sensitive operations.

1.2 Technology Stack

HTML5, CSS3, JavaScript, browser localStorage/sessionStorage, from-scratch RSA, from-scratch ECC, from-scratch SHA-256, and from-scratch HMAC-SHA256.

2. Login and Registration Module

Registration encrypts name, username, email, contact number, and OTP secret before storage. Login decrypts only after HMAC verification and never stores plaintext passwords.

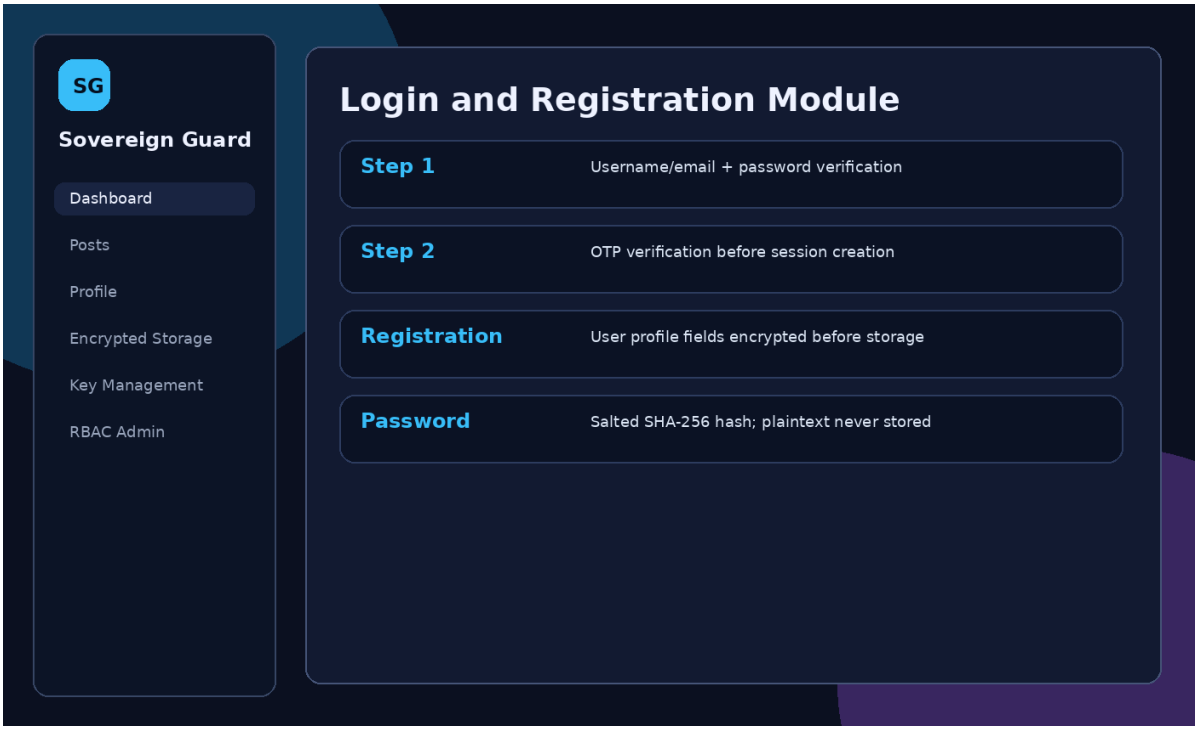

Figure: 2. Login and Registration Module screenshot/diagram.

2.1 Registration Flow

Validate input, generate salt, hash password, generate OTP secret, encrypt user data with RSA, attach HMAC, and store ciphertext.

2.2 Login Flow

Find encrypted identifier, verify HMAC, decrypt username/email, re-hash password with stored salt, verify OTP, and create signed session.

2.3 Implementation Details

Login and registration are implemented in js/app.js with encrypted records and salted hashes.

3. User Data Encryption and Decryption

The system uses two asymmetric algorithms differently: RSA encrypts account/profile fields and OTP secrets, while ECC encrypts social post data.

3.1 Fields Encrypted

User name, username, email, phone, OTP secret, post title, tags, body, and sealed private key records.

3.2 RSA Implementation

Manual prime selection, n = p*q, totient, modular inverse, square-and-multiply modular exponentiation, encryption/decryption, and token signing.

3.3 ECC Implementation

Manual finite-field curve, point addition, point doubling, scalar multiplication, byte-to-point mapping, and EC-ElGamal-style encryption.

3.4 Algorithm Separation

RSA is used for account/profile data; ECC is used for posts; HMAC is used only for integrity, not encryption.

Data Type	Fields	Algorithm
User account	name, username, email, phone	RSA
OTP secret	secret	RSA
Social post	title, tags, body	ECC
Session token	payload signature	RSA
Record integrity	encrypted fields and metadata	HMAC-SHA256

4. Password Hashing and Salting

Passwords are stored only as salted iterative SHA-256 hashes. Each user receives a unique salt stored beside the hash.

4.1 Hashing Algorithm Used

From-scratch SHA-256 implemented in js/crypto.js.

4.2 Salt Generation

Random hexadecimal salt generated per account.

4.3 Verification Process

Retrieve salt, re-hash submitted password, compare with stored hash, then continue to OTP.

5. Two-Factor Authentication (2FA)

Two-step authentication requires both the primary password and a second OTP factor before session creation.

5.1 2FA Method

OTP is generated from a time window and encrypted OTP secret using from-scratch HMAC-SHA256. The classroom demo displays the OTP after password verification.

5.2 Code Snippet

otp(secret) = HMAC(secret, timeWindow) mod 1000000

6. Key Management Module

The KMS generates RSA/ECC keys, distributes active public keys to the encryption layer, stores sealed private key packets, and supports admin key rotation.

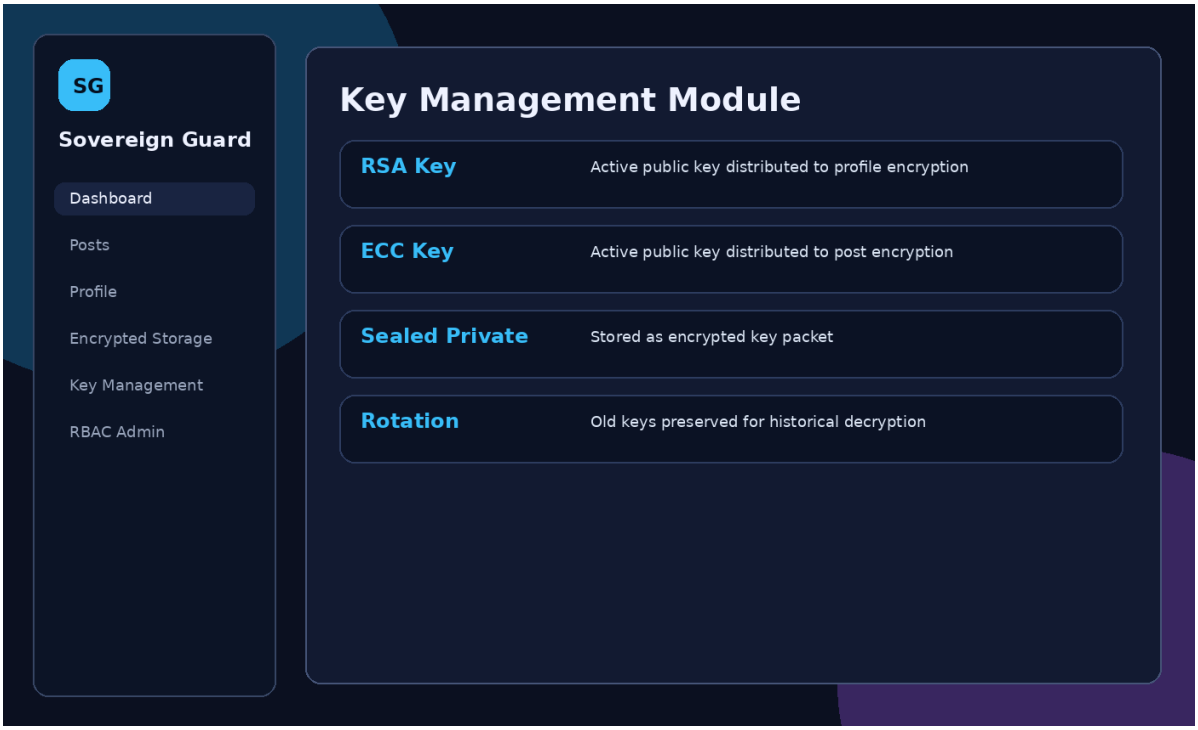

Figure: 6. Key Management Module screenshot/diagram.

6.1 Key Storage Security

Public keys are stored directly, while private key objects are sealed as RSA-encrypted key packets.

6.2 Key Rotation Policy

New keys become active for future writes; old sealed keys remain for decrypting existing records.

7. Post and Profile Management

Users can create, view, and edit posts and update profiles. All profile fields are RSA-encrypted and all post fields are ECC-encrypted before storage.

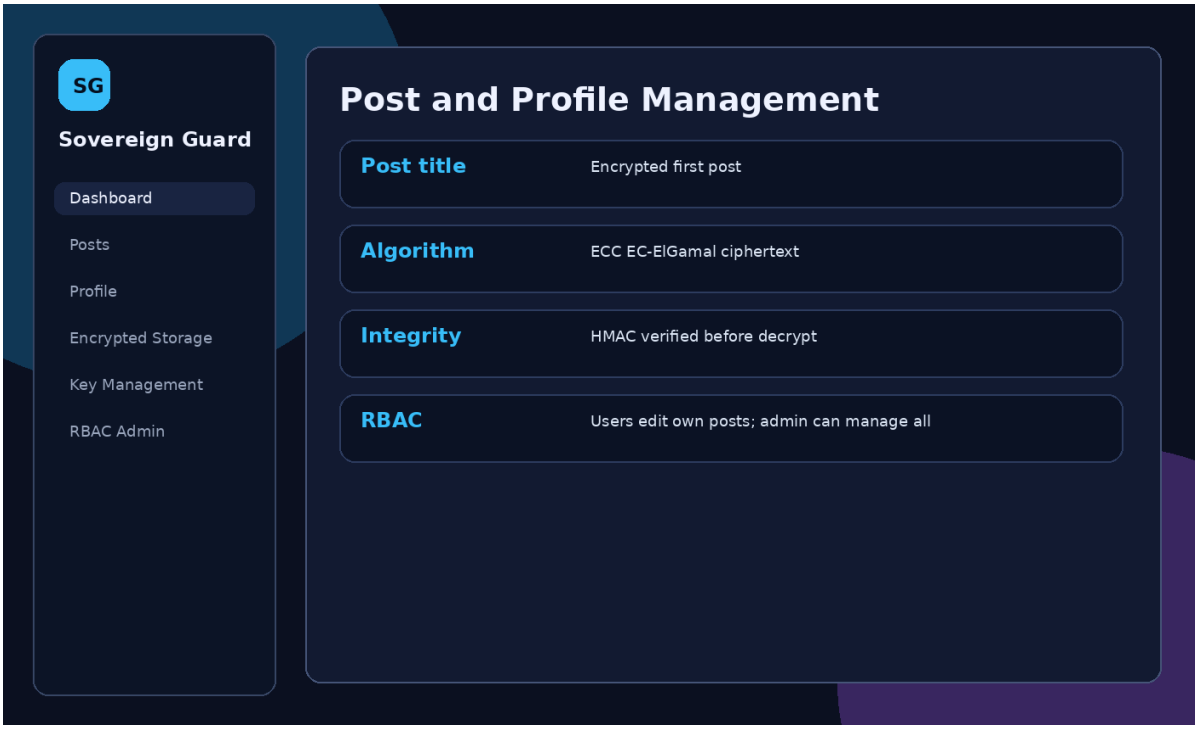

Figure: 7. Post and Profile Management screenshot/diagram.

7.1 Post Module

Create, view, edit, and delete post flows check session and RBAC before encryption/decryption.

7.2 Profile Module

Profile updates are re-encrypted with the active RSA key and a fresh HMAC.

7.3 Screenshots

The project includes screenshots for login, dashboard, post, storage, and key management screens.

8. Data Storage Security

Raw storage contains ciphertext objects, salted password hashes, MACs, sealed keys, and audit logs. Plaintext profile and post values are not directly stored.

Figure: 8. Data Storage Security screenshot/diagram.

8.1 Evidence of Encrypted Storage

The encrypted database page displays RSA/ECC ciphertext records and HMAC tags for instructor verification.

9. Message Authentication Code (MAC)

HMAC-SHA256 verifies encrypted record integrity before decryption and blocks display if unauthorized modification is detected.

9.1 MAC Algorithm Used

From-scratch HMAC-SHA256 built on the project SHA-256 function.

9.2 Integrity Verification Flow

Canonicalize record, compute HMAC, store tag, recompute before reads, and block decryption on mismatch.

10. Role-Based Access Control (RBAC)

RBAC separates administrator and regular user privileges and protects sensitive operations with permission checks.

10.1 Roles Defined

Admin manages keys and all records; regular users manage their own profiles and posts.

10.2 Permission Matrix

Admin can rotate keys, view users, delete any post, and see full audit logs; regular users cannot.

Operation / Resource	Admin	Regular User
View/edit own profile	Yes	Yes
Create/edit own posts	Yes	Yes
Delete any post	Yes	No
View all user accounts	Yes	No
Manage / rotate keys	Yes	No
View audit logs	Full	Limited

11. Secure Session Management

Session tokens are created only after password and OTP verification. Tokens include user ID, role, issue time, expiry time, nonce, RSA signature, and HMAC tag.

11.1 Token Signing / Verification

Each protected action checks token HMAC, RSA signature, expiry, and role authorization.

12. GitHub Repository and Project Structure

Repository URL should be added after upload. The project contains index.html, css/styles.css, js/crypto.js, js/app.js, screenshots, docs, and README.md.

12.1 Repository Structure

sovereign_guard/index.html, css/, js/, screenshots/, docs/, README.md

12.2 README Overview

README includes run instructions, demo accounts, implemented modules, and security disclaimer.

13. Conclusion

Sovereign Guard demonstrates registration, login, 2FA, encrypted profile and post management, MAC integrity checking, key lifecycle management, RBAC, and session protection. The project is educational and uses small parameters; it is not production security software.